8/10/25

Task - 11

Use Tkinter module for UI design

AIM:-
Demonstrate basic operations and understanding of data structure like Binary Search Tree (BST) min/max Heap, Priority Queue, Hashing Collision handling / graph representation, and B-Tree, typically asked in academic contexts.

ALGORITHM:
1. Binary Search Tree (BST) Insertion:
- Start at root
- Compare key with current node; go left if smaller, right if larger
- Insert as leaf node.

- Add element at end
- Heapify-up to maintain min heap property.

- Use queue to visit nodes level by level.

- Hashing key to index
- Handle collision using linked list at index

Output

Form

Name

Jagan

Roll

108

Submit

Name: Jagan RollNo: 108

OK

```
class Node:
        def __init__(self, key):
            self.left = None
             self.right = None
            self.val = key

def insert(root, key):
        if root is None:
                return Node(key)
      else:
        if root.val<key:
            root.right = insert(root.right, key)
       else:
            root.left = insert(root.left, key)
      return root.

#Example Usage
root = Node(50)
root = insert(root, 30)
root = insert(root, 20)
root = insert(root, 40)
root = insert(root, 70)
root = insert(root, 60)
root = insert(root, 80)

#Inorder traversal: 20 30 40 50 60 70 80.
```

OUTPUT:-

['B','C']

Min Heap Example

```
import heapq
heap = [ ]
heapq.heappush(heap, 10)
heapq.heappush(heap, 5)
heapq.heappush(heap, 20)
print(heapq.heappop(heap)).
```

Graph (Adjacency List) Example

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
print(graph['A'])
```

VEL TECH-CSE	
EX NO.	11
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	2
TOTAL (20)	15
SIGN WITH DATE	15/10/25

RESULT:

Thus, the implemation and verification of the data structures is completed sucessfully.

15/10/25